

ANEXO I

C.I.F.P. Cesar Manrique

C.F.G.S. Administración de Sistemas Informáticos y en Red

ANTEPROYECTO PARA SOLICITAR LA REALIZACIÓN DEL MÓDULO PROYECTO

Nombre del proyecto: APP gestión de economía doméstica

Alumno: Abraham Quintana Micó

Curso: 3º DAW Semipresencial

Tutor: José David Díaz Díaz

OBJETIVOS:

El objetivo de este proyecto es desarrollar una aplicación web que permita a los usuarios gestionar sus gastos personales de forma sencilla y visual. La solución ofrecerá un sistema para registrar ingresos y gastos, calcular el balance disponible y visualizar la evolución financiera mediante gráficos interactivos.

El sistema permitirá a los usuarios:

- Registrar, modificar y eliminar gastos e ingresos.
- Consultar el balance total disponible.
- Visualizar la evolución del saldo a lo largo del tiempo mediante gráficos.
- Analizar la distribución de gastos por categorías.
- Acceder a una interfaz clara y responsive desde cualquier dispositivo.

El proyecto se utilizará como demostración técnica de un sistema web completo basado en tecnologías modernas, orientado a la gestión financiera personal.

PREANÁLISIS DE LO EXISTENTE

Actualmente, muchas personas gestionan sus gastos personales mediante hojas de cálculo, notas en el móvil o incluso de forma mental, lo que puede provocar falta de control, errores de cálculo o pérdida de información.

Existen aplicaciones comerciales de gestión financiera, pero muchas incluyen funciones avanzadas que resultan innecesarias para usuarios que solo buscan un control básico de ingresos y gastos. Además, algunas requieren suscripciones o acceso a datos bancarios, lo que puede generar desconfianza.

Este proyecto busca ofrecer una alternativa sencilla, accesible y totalmente controlada por el usuario, sin depender de servicios externos ni compartir datos sensibles.

PREANÁLISIS DEL SISTEMA

*El sistema estará compuesto por varios módulos claramente diferenciados, orientados a garantizar una arquitectura mantenible, segura y escalable. El desarrollo se realizará siguiendo una metodología **TDD (Test-Driven Development)**, lo que permitirá asegurar la calidad del código desde las primeras fases del proyecto mediante la creación de pruebas unitarias y de integración antes de implementar la lógica correspondiente.*

1. Backend con API REST segura

El backend se desarrollará con **Spring Boot**, implementando una API REST que gestionará todas las operaciones relacionadas con gastos, ingresos y estadísticas. Se incluirá un sistema de **autenticación y autorización basado en JWT**, permitiendo que cada usuario acceda únicamente a sus propios datos.

Las principales funcionalidades serán:

- CRUD de gastos e ingresos.
- Cálculo del balance disponible.
- Endpoints para estadísticas:
 - Gastos agrupados por categoría.
 - Evolución del saldo a lo largo del tiempo.
- Gestión de usuarios y autenticación JWT.
- Validación de datos y manejo centralizado de errores.

Dependencias principales de Spring Boot:

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- spring-boot-starter-security
- jjwt
- postgresql (driver JDBC)

2. Base de datos

El sistema almacenará toda la información en una base de datos **PostgreSQL**, incluyendo:

- Transacciones (gastos e ingresos).
- Categorías.
- Usuarios.
- Datos agregados para estadísticas.

3. Frontend interactivo

El frontend se desarrollará con React, utilizando la librería de componentes Material UI para garantizar una interfaz moderna, accesible y consistente.

Las funcionalidades principales serán:

- Formulario para registrar gastos e ingresos.
- Listado filtrable de transacciones.
- Dashboard con gráficas interactivas:
 - Gráfico de líneas para evolución del saldo.
 - Gráfico circular o de barras para distribución por categorías.
- Gestión de sesión mediante JWT almacenado de forma segura.
- Comunicación con el backend mediante Axios.

4. Despliegue y contenedores

El sistema se desplegará mediante Docker, utilizando contenedores independientes para:

- Frontend
- Backend

- Base de datos PostgreSQL

La orquestación se realizará con Docker Compose, permitiendo levantar todo el entorno con un único comando.

PREDISEÑO DEL SISTEMA

El sistema se diseñará siguiendo una arquitectura cliente–servidor, con una clara separación entre frontend, backend y base de datos, y con todos los componentes desplegados en contenedores Docker. Se priorizará la simplicidad, la mantenibilidad y la seguridad mediante autenticación basada en tokens JWT.

Backend

El backend se desarrollará con Java y Spring Boot, exponiendo una API REST que gestionará todas las operaciones relacionadas con usuarios, gastos, ingresos y estadísticas.

Se utilizarán, entre otras, las siguientes dependencias:

- **spring-boot-starter-web**: para la creación de la API REST.
- **spring-boot-starter-data-jpa**: para el acceso a datos mediante JPA.
- **spring-boot-starter-validation**: para la validación de datos de entrada.
- **spring-boot-starter-security**: para la configuración de seguridad.
- **Librería JWT**: para la generación y validación de tokens.
- **postgresql**: driver JDBC para la base de datos.

El backend implementará:

- Gestión de usuarios y autenticación mediante JWT.
- CRUD de gastos e ingresos.
- Endpoints para estadísticas (por categoría y por evolución temporal).
- Manejo centralizado de errores y validaciones.

Base de datos

La base de datos será **PostgreSQL**, donde se almacenarán:

- Usuarios.
- Transacciones (gastos e ingresos).
- Categorías.

Se utilizará Spring Data JPA para mapear las entidades y gestionar las operaciones de persistencia. La base de datos se ejecutará en un contenedor independiente.

Frontend

El frontend se desarrollará con React, utilizando:

- **Material UI** como librería de componentes para construir una interfaz moderna y consistente.
- **Axios** para la comunicación con la API REST.
- Una librería de gráficos **Recharts** para representar:
 - La evolución del saldo en el tiempo.
 - La distribución de gastos por categorías.

La aplicación permitirá:

- Autenticación de usuarios mediante JWT.
- Registro y gestión de gastos e ingresos.
- Visualización de un dashboard con gráficas interactivas.
- Filtros por fecha y categoría.

Infraestructura y despliegue

El sistema se desplegará mediante **Docker** y **Docker Compose**, definiendo al menos tres servicios:

- Contenedor para el backend (Spring Boot).
- Contenedor para el frontend (React).
- Contenedor para PostgreSQL.

Docker Compose gestionará la red interna entre servicios y las variables de entorno necesarias (credenciales de base de datos, URLs del backend, etc.).

El repositorio de código se gestionará con **Git** y **GITHUB** como repositorio remoto.

ESTIMACIÓN DE COSTES

Coste temporal

El desarrollo se ajustará a un plazo realista de un mes:

1. Planificación, diseño y configuración inicial: 3–4 días
2. Desarrollo del backend con TDD (API REST + seguridad JWT + BD): 1,5 semanas
3. Desarrollo del frontend (UI con Material UI + dashboard + autenticación): 1 semana
4. Integración, pruebas completas y ajustes finales: 3–4 días
5. Documentación y preparación de la presentación: 3–4 días

Total estimado: 4 semanas

Coste económico

El proyecto se desarrollará íntegramente con herramientas de código abierto, por lo que no requiere inversión en licencias.

Infraestructura y hosting (opcional):

- Backend y base de datos: plataformas gratuitas como Railway, Render o Supabase.
- Frontend: Vercel o Netlify (gratuitos).
- Dominio web: opcional (~10–15 €/año).
- Certificado SSL: gratuito mediante Let's Encrypt.

Costes de desarrollo:

- Desarrollo completo con seguridad y dashboard: 5000–8000 €.
- Mantenimiento mensual: ~50–100 €.

